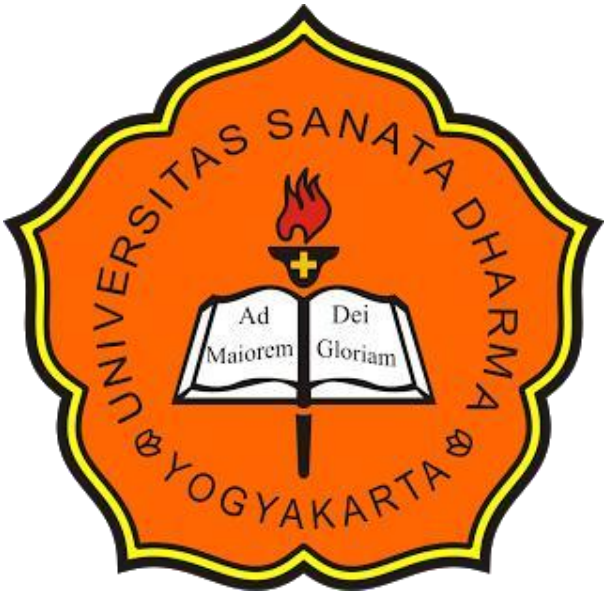
LAPORAN HASIL PRAKTIKUM 6

Pemrograman Berbasis Web



Oleh

Nama : Maria Rufina Editha Kedhi
NIM 255314097

PROGRAM STUDI INFORMATIKA

FAKULTAS SAINS DAN TEKNOLOGI

UNIVERSITAS SANATA DHARMA

YOGYAKARTA

2026

A. Tujuan

- Menyiapkan proyek React dengan Vite dan menjalankannya di browser.
- Menulis JSX serta membangun komponen fungsi yang dapat digunakan kembali.
- Mengirim data melalui props dan memperbarui antarmuka melalui state.
- Menangani event, form, tampilan bersyarat, dan daftar dengan key stabil.
- Mengembangkan aplikasi sederhana dan menguji perilakunya secara sistematis.

B. Ringkasan Konsep

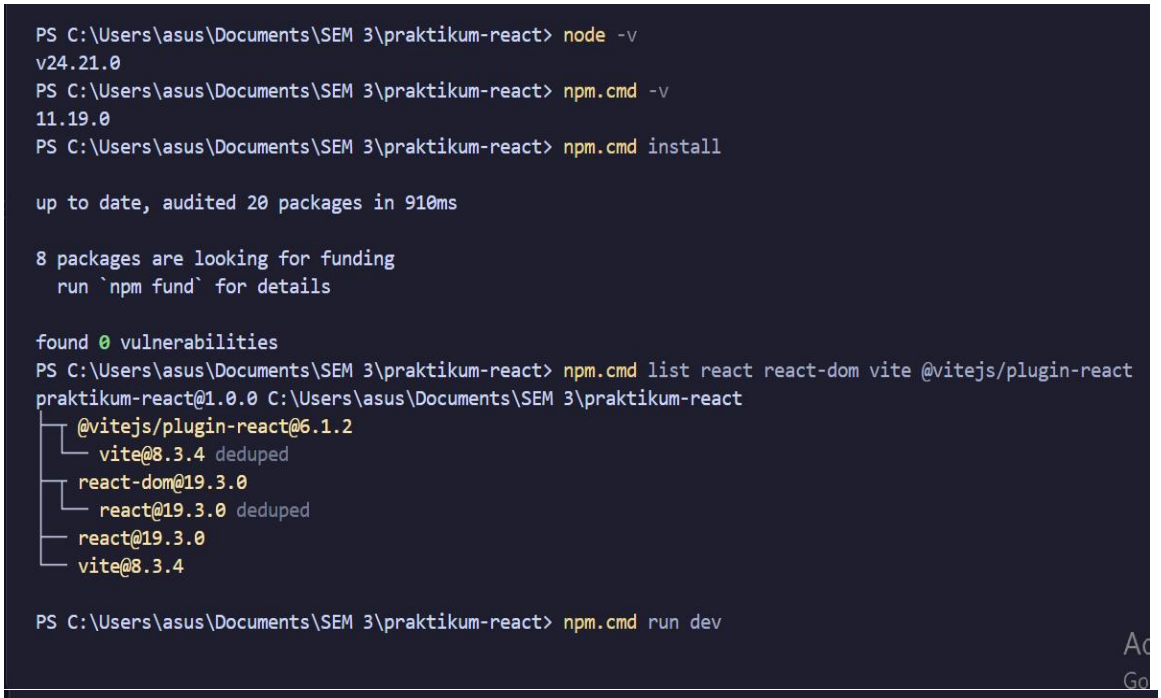
React adalah library JavaScript yang digunakan untuk membangun antarmuka pengguna berbasis komponen. Dalam praktikum ini, JSX digunakan untuk menuliskan struktur tampilan dengan sintaks yang mirip HTML di dalam JavaScript. Komponen digunakan untuk membagi tampilan menjadi bagian-bagian kecil agar kode lebih terstruktur dan mudah dikelola.

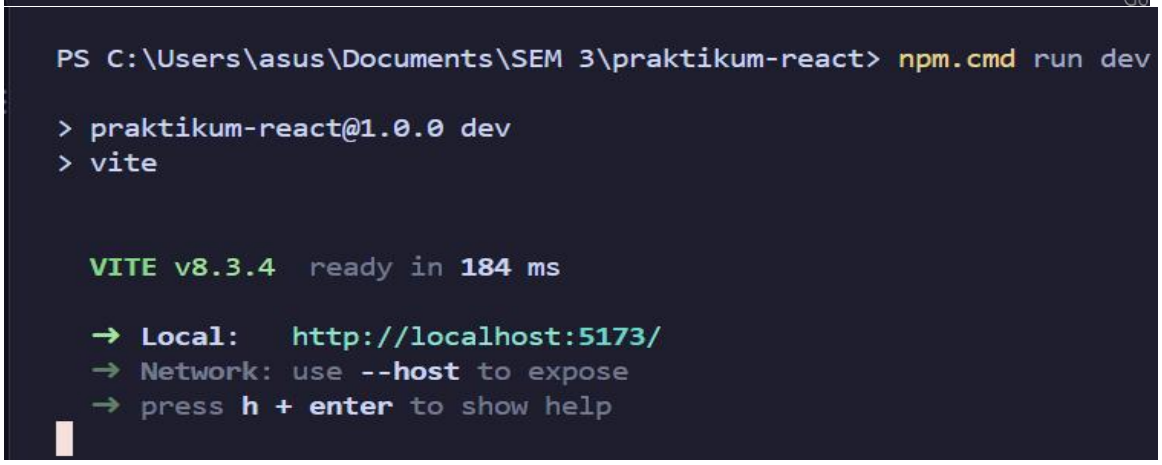
Props digunakan untuk mengirimkan data dari komponen induk ke komponen anak, misalnya mengirimkan nama, NIM, dan program studi ke komponen profil mahasiswa. State digunakan untuk menyimpan data yang dapat berubah, seperti jumlah kehadiran dan daftar tugas. Event handling digunakan untuk menjalankan fungsi ketika pengguna melakukan tindakan, misalnya menekan tombol tambah, hapus, atau edit tugas.

Pada daftar mata kuliah dan daftar tugas, key digunakan untuk memberikan identitas unik pada setiap elemen yang ditampilkan melalui proses map(). Praktikum ini juga menerapkan localStorage untuk menyimpan daftar tugas di browser agar data tidak hilang ketika halaman dimuat ulang. Selain itu, fitur pengeditan tugas menggunakan validasi input untuk mencegah pengguna menyimpan judul tugas yang kosong.

C. Versi Node dan Dependensi

Komponen	Versi
Node.js	24.21.0
npm	11.19.0
React	19.3.0
React DOM	19.3.0
Vite	8.3.4
Plugin React Vite	6.1.2





Penjelasan

Praktikum ini menggunakan Node.js versi 24.21.0 dan npm versi 11.19.0 sebagai lingkungan pengembangan. Library React dan React DOM yang digunakan adalah versi 19.3.0, sedangkan Vite versi 8.3.4 digunakan sebagai alat bantu untuk menjalankan server pengembangan. Selain itu, proyek menggunakan plugin React Vite versi 6.1.2 untuk mendukung penggunaan React. Seluruh dependensi berhasil dipasang menggunakan perintah npm install, dan server pengembangan berhasil dijalankan pada alamat lokal http://localhost:5173/.

D. Langkah Kerja

- Menyiapkan lingkungan pengembangan

Membuka Visual Studio Code dan membuka folder proyek praktikum-react. Selanjutnya, memeriksa versi Node.js dan npm melalui terminal menggunakan perintah node -v dan npm.cmd -v.

- Memasang dependensi

Menjalankan perintah npm.cmd install pada terminal untuk memasang dependensi yang dibutuhkan oleh proyek React. Setelah proses selesai, memastikan instalasi berhasil tanpa error.

- Memeriksa dependensi proyek

Menjalankan perintah npm.cmd list react react-dom vite @vitejs/plugin-react untuk mengetahui versi React, React DOM, Vite, dan plugin React Vite yang digunakan.

- Menjalankan aplikasi React

Menjalankan perintah npm.cmd run dev untuk mengaktifkan server pengembangan Vite. Setelah server berhasil dijalankan, membuka alamat http://localhost:5173/ melalui browser untuk melihat tampilan aplikasi.

- Membuat komponen profil mahasiswa

Membuat komponen ProfilMahasiswa.jsx untuk menampilkan nama, NIM, dan program studi mahasiswa. Data dikirimkan dari komponen utama menggunakan props.

- Membuat daftar mata kuliah

Membuat komponen MataKuliah.jsx yang menampilkan minimal empat mata kuliah dalam bentuk kartu. Data ditampilkan menggunakan fungsi map() dan setiap elemen diberikan key yang unik.

- Membuat penghitung kehadiran

Membuat komponen PenghitungKehadiran.jsx dengan fitur tambah, kurang, dan reset. Jumlah kehadiran dibatasi dari 0 hingga 30 menggunakan state dan event handling.

- Membuat daftar tugas

Membuat komponen DaftarTugas.jsx yang memungkinkan pengguna menambahkan tugas, menandai tugas selesai, serta memfilter tugas berdasarkan kategori semua, aktif, dan selesai.

- Menambahkan penyimpanan data

Menggunakan localStorage untuk menyimpan daftar tugas di browser. Data dibaca ketika aplikasi pertama kali dibuka dan diperbarui ketika daftar tugas berubah, sehingga data tetap tersedia setelah halaman dimuat ulang.

- Menambahkan fitur edit tugas

Menambahkan fitur untuk mengubah judul tugas dengan validasi agar judul tidak boleh kosong. Identitas tugas tetap dipertahankan ketika judul diubah.

- Melakukan pengujian aplikasi

Menguji setiap fitur, termasuk tombol tambah, kurang, dan reset pada penghitung kehadiran; filter daftar tugas; penyimpanan data setelah halaman dimuat ulang; serta pengeditan tugas yang sudah selesai.

- Mendokumentasikan hasil praktikum

Mengambil screenshot tampilan aplikasi dan hasil pengujian, kemudian menyusun laporan yang berisi tujuan, ringkasan konsep, versi Node.js dan dependensi, langkah kerja, potongan kode penting, hasil pengujian, analisis, dan kesimpulan.

E. Potongan Kode

App.css

```css
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: Arial, sans-serif;
  background-color: #f3f5f9;
  color: #252b36;
}
```

```css
.container {
  width: 90%;
  max-width: 900px;
  margin: 30px auto;
```

```
}

h1,
h2 {
  color: #263b63;
}

h1 {
  margin-bottom: 8px;
}

h2 {
  margin-top: 0;
}

h3 {
  color: #34496e;
}

.subjudul {
  color: #687386;
  margin-bottom: 25px;
}

.kartu {
  background-color: white;
  padding: 22px;
  margin-bottom: 22px;
  border-radius: 12px;
  box-shadow: 0 3px 10px rgba(0, 0, 0, 0.06);
}

.daftar-kartu {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 15px;
}

.kartu-kuliah {
  padding: 15px;
  background-color: #f4f7fc;
  border: 1px solid #e1e7f0;
  border-radius: 9px;
}

.kartu-kuliah h3 {
  margin-top: 0;
}

.angka-kehadiran {
  font-size: 48px;
  font-weight: bold;
  text-align: center;
  color: #365da8;
  margin: 15px 0;
}

button {
```

```
  padding: 10px 14px;
  margin: 5px 5px 5px 0;
  border: none;
  border-radius: 6px;
  background-color: #365da8;
  color: white;
  cursor: pointer;
}

button:hover {
  background-color: #263b63;
}

button:focus-visible,
input:focus-visible {
  outline: 2px solid #365da8;
  outline-offset: 2px;
}

input {
  width: 100%;
  max-width: 400px;
  padding: 11px;
  border: 1px solid #cbd2de;
  border-radius: 6px;
  font-size: 14px;
}

.form-tugas {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  gap: 5px;
  margin-top: 8px;
}

.filter {
  margin: 10px 0;
}

.filter-aktif {
  background-color: #172b50;
  outline: 2px solid #9aafd6;
}

.daftar-tugas {
  padding-left: 20px;
}

.item-tugas {
  display: flex;
  justify-content: space-between;
  align-items: center;
  flex-wrap: wrap;
  gap: 10px;
  padding: 12px 0;
  border-bottom: 1px solid #e6e9ef;
}
```

```css
.selesai {
  color: #7b8492;
  text-decoration: line-through;
}

.aksi {
  display: flex;
  flex-wrap: wrap;
  gap: 5px;
}

.aksi button {
  margin: 0;
}

.pesan {
  padding: 10px;
  background-color: #eef3fc;
  border-radius: 6px;
  color: #263b63;
}

.ringkasan {
  margin-top: 20px;
  padding: 12px;
  background-color: #f4f7fc;
  border-radius: 8px;
}

.ringkasan p {
  margin: 6px 0;
}

@media (max-width: 600px) {
  .container {
    width: 94%;
    margin: 18px auto;
  }

  .kartu {
    padding: 16px;
  }

  .item-tugas {
    align-items: flex-start;
    flex-direction: column;
  }

  .form-tugas {
    align-items: stretch;
    flex-direction: column;
  }

  .form-tugas button {
    width: 100%;
  }
}
```

App.jsx

```
import React from 'react';
import ProfilMahasiswa from './ProfilMahasiswa.jsx';
import MataKuliah from './MataKuliah.jsx';
import PenghitungKehadiran from './PenghitungKehadiran.jsx';
import DaftarTugas from './DaftarTugas.jsx';

function App() {
  const nama = 'Maria Rufina Editha';
  const nim = '255314097';
  const prodi = 'Informatika';

  return (
    <main className="container">
      <h1>Praktikum React</h1>
      <p className="subjudul">
        Implementasi JSX, Props, State, dan localStorage
      </p>

      <ProfilMahasiswa nama={nama} nim={nim} prodi={prodi} />
      <MataKuliah />
      <PenghitungKehadiran />
      <DaftarTugas />
    </main>
  );
}

export default App;
```

DaftarTugas.jsx

```
import React, { useState, useEffect } from 'react';

const KUNCI_STORAGE = 'daftar-tugas-kuliah';

function bacaTugas() {
  try {
    const data = localStorage.getItem(KUNCI_STORAGE);

    if (data === null) {
      return [];
    }

    const hasil = JSON.parse(data);

    if (!Array.isArray(hasil)) {
      return [];
    }

    return hasil.filter(
      (tugas) =>
        tugas !== null &&
        typeof tugas === 'object' &&
        Number.isSafeInteger(tugas.id) &&
        typeof tugas.judul === 'string' &&
        typeof tugas.selesai === 'boolean'
    );
  } catch (error) {
```

```
    return [];
  }
}
```

```
function DaftarTugas() {
  // Lazy initializer membaca data hanya saat state pertama dibuat.
  const [daftarTugas, setDaftarTugas] = useState(bacaTugas);
  const [judul, setJudul] = useState('');
  const [filter, setFilter] = useState('Semua');
  const [idEdit, setIdEdit] = useState(null);
  const [pesan, setPesan] = useState('');
```

```
  // Menyimpan data ketika daftar tugas berubah.
  useEffect(() => {
    try {
      localStorage.setItem(
        KUNCI_STORAGE,
        JSON.stringify(daftarTugas)
      );
    } catch (error) {
      console.error('Data tugas gagal disimpan:', error);
    }
  }, [daftarTugas]);
```

```
  function simpanTugas(e) {
    e.preventDefault();
    const judulBaru = judul.trim();
```

```
    if (judulBaru === '') {
      setPesan('Judul tugas tidak boleh kosong.');
      return;
    }
```

```
    if (idEdit !== null) {
      setDaftarTugas((daftarSebelumnya) =>
        daftarSebelumnya.map((tugas) =>
          tugas.id === idEdit
            ? { ...tugas, judul: judulBaru }
            : tugas
        )
      );
      setPesan('Judul tugas berhasil diperbarui.');
      setIdEdit(null);
    } else {
      const tugasBaru = {
        id: Date.now(),
        judul: judulBaru,
        selesai: false
      };
```

```
      setDaftarTugas((daftarSebelumnya) => [
        ...daftarSebelumnya,
        tugasBaru
      ]);
      setPesan('Tugas berhasil ditambahkan.');
    }
```

```
    setJudul('');
```

```
}
```

```
function ubahStatus(id) {
  setDaftarTugas((daftarSebelumnya) =>
    daftarSebelumnya.map((tugas) =>
      tugas.id === id
        ? { ...tugas, selesai: !tugas.selesai }
        : tugas
    )
  );
  setPesan('');
}
```

```
function mulaiEdit(tugas) {
  setIdEdit(tugas.id);
  setJudul(tugas.judul);
  setPesan('');
}
```

```
function batalEdit() {
  setIdEdit(null);
  setJudul('');
  setPesan('');
}
```

```
const tugasDitampilkan = daftarTugas.filter((tugas) => {
  if (filter === 'Aktif') return !tugas.selesai;
  if (filter === 'Selesai') return tugas.selesai;
  return true;
});
```

```
const jumlahSelesai = daftarTugas.filter(
  (tugas) => tugas.selesai
).length;
```

```
return (
  <section className="kartu">
    <h2>Daftar Tugas Kuliah</h2>
```

```
    <form onSubmit={simpanTugas}>
      <label htmlFor="judul-tugas">Judul tugas</label>
      <div className="form-tugas">
        <input
          id="judul-tugas"
          type="text"
          value={judul}
          onChange={(e) => setJudul(e.target.value)}
          placeholder="Contoh: Praktikum React"
        />
        <button type="submit">
          {idEdit !== null ? 'Simpan Edit' : 'Tambah Tugas'}
        </button>
        {idEdit !== null && (
          <button type="button" onClick={batalEdit}>
            Batal
          </button>
        )}
      </div>
```

```
    </form>
```

```
    {pesan !== '' && (
      <p className="pesan" role="status">{pesan}</p>
    )}
```

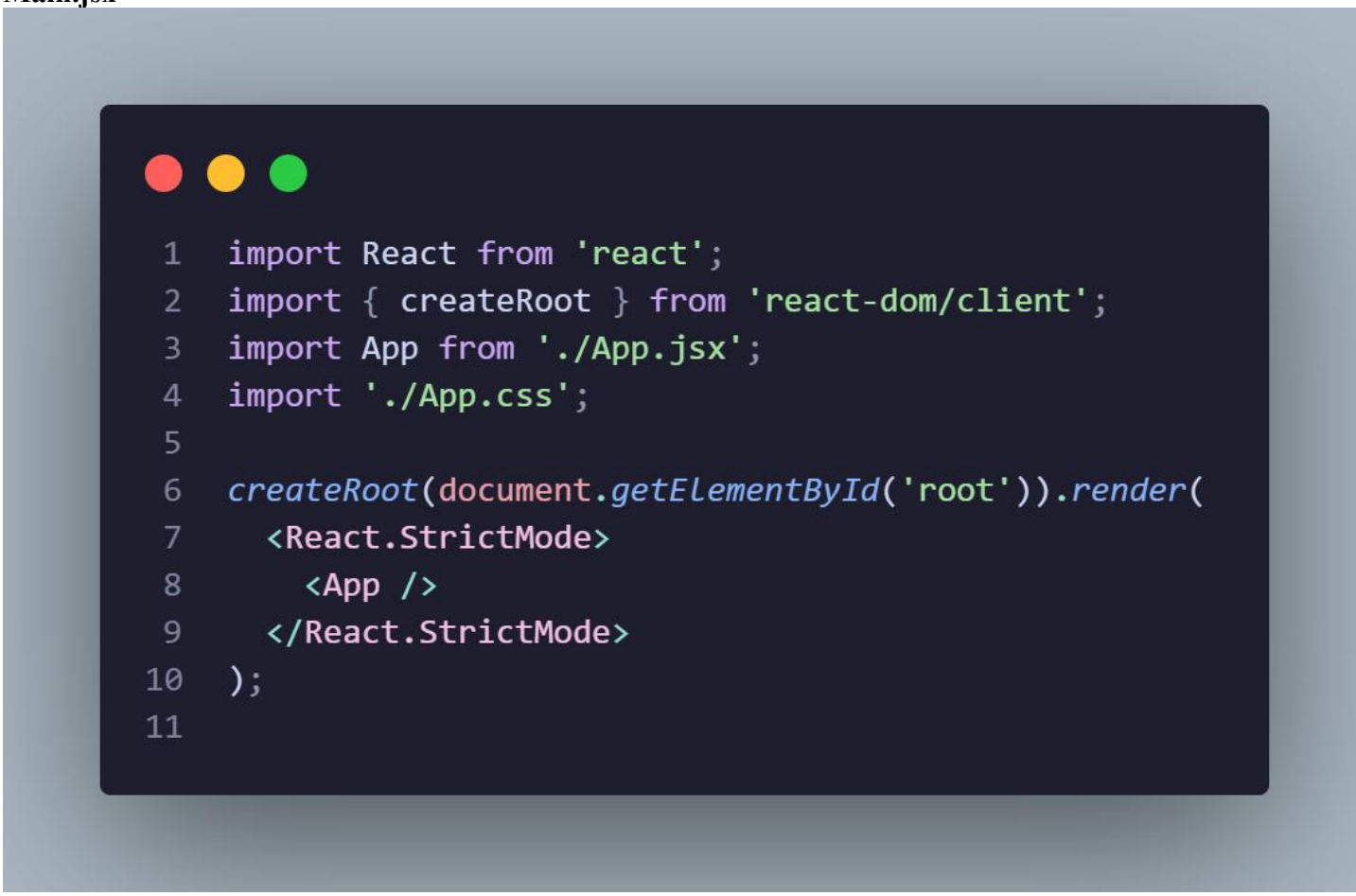
```
    <h3>Filter Tugas</h3>
    <div className="filter">
      {['Semua', 'Aktif', 'Selesai'].map((kategori) => (
        <button
          key={kategori}
          type="button"
          className={filter === kategori ? 'filter-aktif' : ''}
          aria-pressed={filter === kategori}
          onClick={() => setFilter(kategori)}
        >
          {kategori}
        </button>
      ))}
    </div>
```

```
    {tugasDitampilkan.length === 0 ? (
      <p>Belum ada tugas pada kategori ini.</p>
    ) : (
      <ul className="daftar-tugas">
        {tugasDitampilkan.map((tugas) => (
          <li key={tugas.id} className="item-tugas">
            <span className={tugas.selesai ? 'selesai' : ''}>
              {tugas.judul}
            </span>
            <div className="aksi">
              <button
                type="button"
                onClick={() => ubahStatus(tugas.id)}
              >
                {tugas.selesai ? 'Aktifkan' : 'Selesaikan'}
              </button>
              <button
                type="button"
                onClick={() => mulaiEdit(tugas)}
              >
                Edit
              </button>
            </div>
          </li>
        ))}
      </ul>
    )}
```

```
    <div className="ringkasan">
      <p>Total tugas: {daftarTugas.length}</p>
      <p>Tugas aktif: {daftarTugas.length - jumlahSelesai}</p>
      <p>Tugas selesai: {jumlahSelesai}</p>
    </div>
  </section>
);
}
```

```
export default DaftarTugas;
```

Main.jsx



```jsx
import React from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.jsx';
import './App.css';

createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

MataKuliah.jsx

```jsx
import React from 'react';

function KartuMataKuliah({ nama, kode, sks }) {
 return (
  <article className="kartu-kuliah">
   <h3>{nama}</h3>
   <p>Kode: {kode}</p>
   <p>SKS: {sks}</p>
  </article>
 );
}
```

```jsx
function MataKuliah() {
 const daftarKuliah = [
  { id: 1, nama: 'Pemrograman Web', kode: 'IF201', sks: 3 },
  { id: 2, nama: 'Basis Data', kode: 'IF202', sks: 3 },
  { id: 3, nama: 'Struktur Data', kode: 'IF203', sks: 3 },
  { id: 4, nama: 'Jaringan Komputer', kode: 'IF204', sks: 3 }
 ];
```

```jsx
 return (
  <section className="kartu">
   <h2>Daftar Mata Kuliah</h2>
   <div className="daftar-kartu">
```

```
      {daftarKuliah.map((kuliah) => (
        <KartuMataKuliah
          key={kuliah.id}
          nama={kuliah.nama}
          kode={kuliah.kode}
          sks={kuliah.sks}
        />
      ))}
    </div>
  </section>
 );
}

export default MataKuliah;
```

PenghitungKehadiran.jsx

```
import React, { useState } from 'react';

function PenghitungKehadiran() {
 const [jumlah, setJumlah] = useState(0);

 function tambah() {
  setJumlah((nilai) => Math.min(30, nilai + 1));
 }

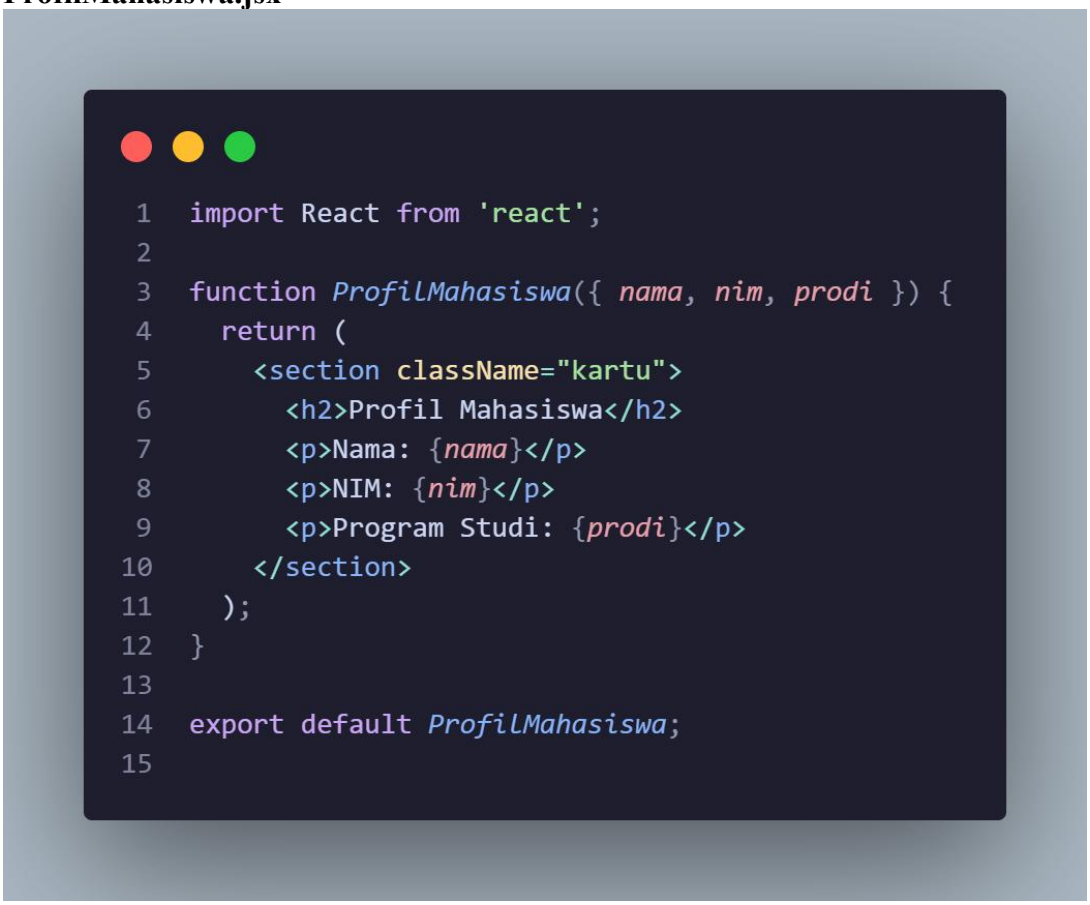
 function kurangi() {
  setJumlah((nilai) => Math.max(0, nilai - 1));
 }

 function reset() {
  setJumlah(0);
 }

 return (
  <section className="kartu">
   <h2>Penghitung Kehadiran</h2>
   <div className="angka-kehadiran">{jumlah}</div>
   <p>Batas nilai: 0 sampai 30</p>
   <div className="aksi">
    <button onClick={kurangi}>Kurangi</button>
    <button onClick={tambah}>Tambah</button>
    <button onClick={reset}>Reset</button>
   </div>
  </section>
 );
}

export default PenghitungKehadiran;
```

ProfilMahasiswa.jsx



```jsx
import React from 'react';

function ProfilMahasiswa({ nama, nim, prodi }) {
  return (
    <section className="kartu">
      <h2>Profil Mahasiswa</h2>
      <p>Nama: {nama}</p>
      <p>NIM: {nim}</p>
      <p>Program Studi: {prodi}</p>
    </section>
  );
}

export default ProfilMahasiswa;

```

Index.html



```html
<!doctype html>
<html lang="id">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Praktikum React</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>

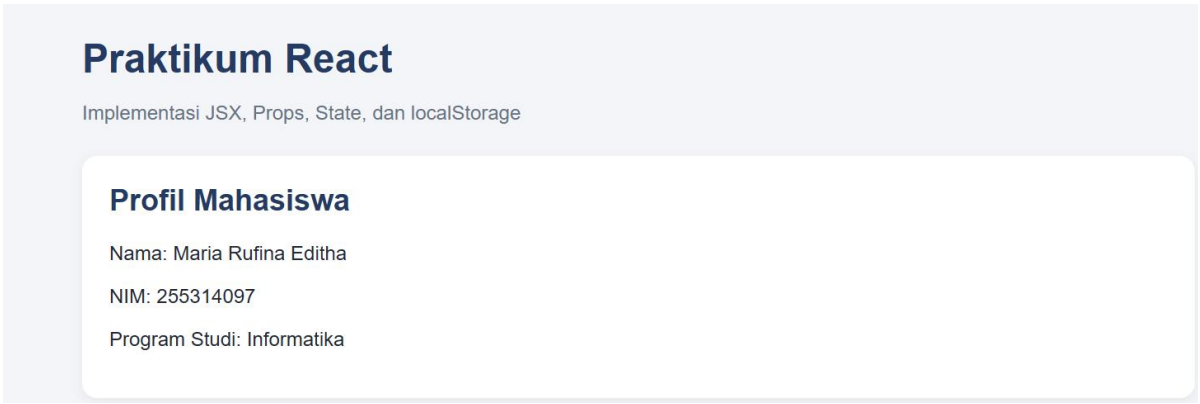
```

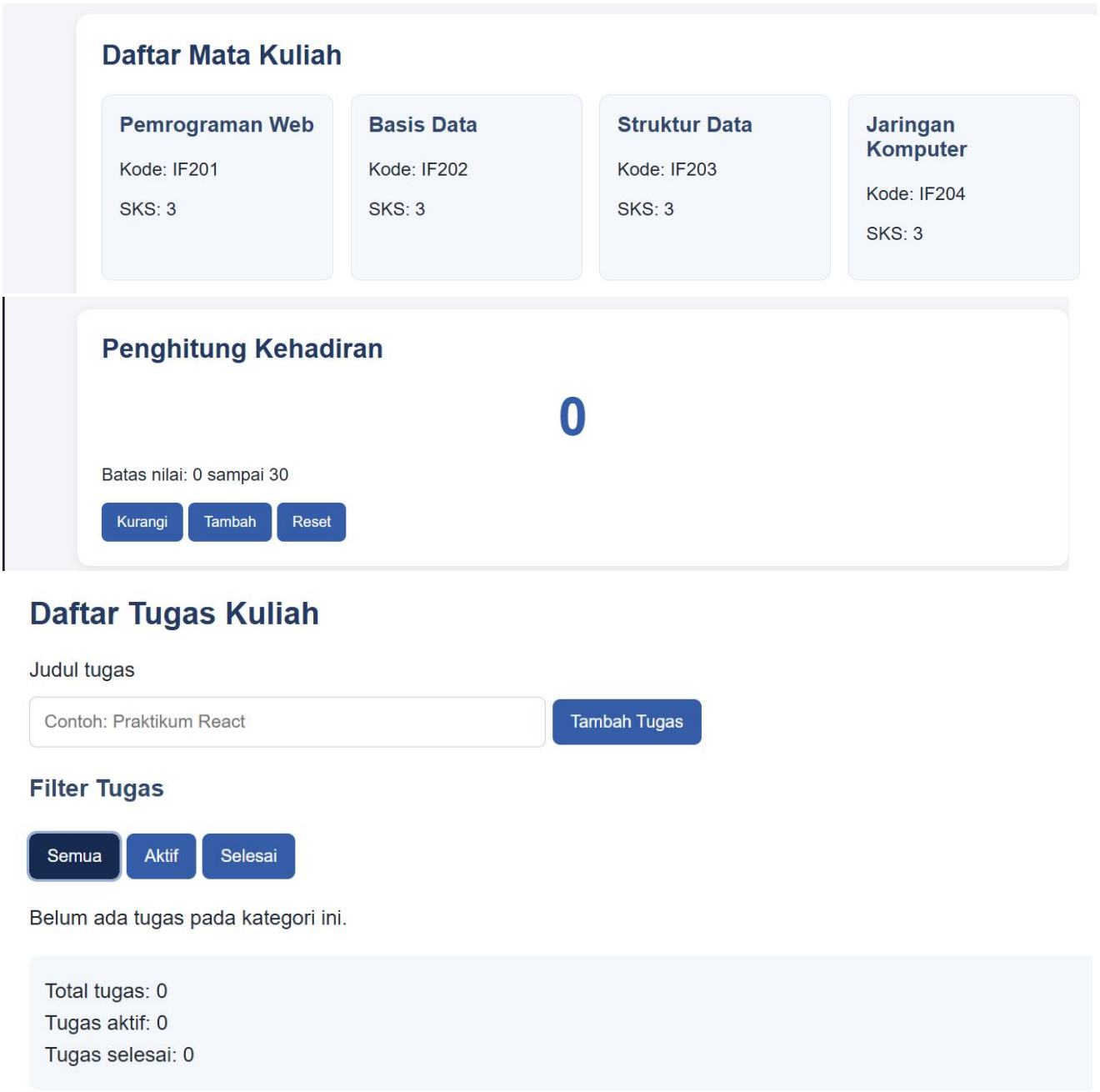
Package.json



```
{
  "name": "praktikum-react",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "@vitejs/plugin-react": "latest",
    "vite": "latest",
    "react": "latest",
    "react-dom": "latest"
  },
  "devDependencies": {}
}
```

F. Hasil Uji





G. Analisis

Berdasarkan praktikum yang dilakukan, React dapat digunakan untuk membangun aplikasi web yang terdiri dari beberapa komponen dengan fungsi berbeda. Penggunaan JSX mempermudah penulisan tampilan, sedangkan props memungkinkan pengiriman data dari komponen utama ke komponen lainnya. Penggunaan komponen juga membuat kode lebih terstruktur dan mudah dikelola.

Pada fitur profil mahasiswa dan daftar mata kuliah, data ditampilkan menggunakan komponen React. Penggunaan key pada daftar mata kuliah membantu React mengenali setiap elemen secara unik. Pada fitur penghitung kehadiran, useState digunakan untuk menyimpan nilai yang berubah ketika tombol ditekan. Pembatasan nilai dari 0 hingga 30 memastikan jumlah kehadiran tidak melebihi batas yang ditentukan.

Pada fitur daftar tugas, state digunakan untuk mengelola data tugas dan pilihan filter. Filter memungkinkan pengguna melihat tugas berdasarkan statusnya tanpa mengubah data utama. Penyimpanan menggunakan

localStorage membuat daftar tugas tetap tersedia setelah halaman dimuat ulang. Selain itu, fitur edit tugas dilengkapi validasi agar judul tidak boleh kosong dan identitas tugas tetap dipertahankan ketika judul diubah.

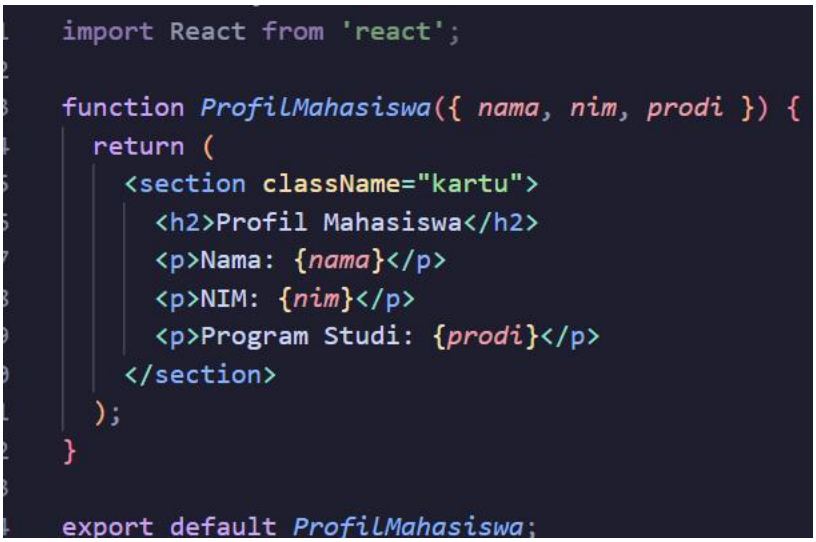
Melalui praktikum ini, dapat dipahami bahwa penggunaan JSX, komponen, props, state, event handling, key, dan localStorage membantu membangun aplikasi web yang interaktif dan terstruktur. Pengujian setiap fitur diperlukan untuk memastikan aplikasi berjalan sesuai dengan kebutuhan. Apabila ditemukan fitur yang belum berjalan dengan baik, kode perlu diperiksa dan diperbaiki sebelum aplikasi dikumpulkan.

E. Penjelasan

1. Props

Props berfungsi untuk mengirimkan data dari komponen induk ke komponen anak. Pada proyek ini, props digunakan untuk mengirimkan informasi mahasiswa seperti nama, NIM, dan program studi ke komponen profil mahasiswa.

Contoh kode dari ProfilMahasiswa.jsx:



```jsx
import React from 'react';

function ProfilMahasiswa({ nama, nim, prodi }) {
  return (
    <section className="kartu">
      <h2>Profil Mahasiswa</h2>
      <p>Nama: {nama}</p>
      <p>NIM: {nim}</p>
      <p>Program Studi: {prodi}</p>
    </section>
  );
}

export default ProfilMahasiswa;
```

Komponen ProfilMahasiswa menerima data melalui props nama, nim, dan prodi, kemudian menampilkannya pada halaman. Dengan props, data dapat dikirim dari komponen utama tanpa harus menuliskannya ulang di setiap komponen.

2. State

State berfungsi untuk menyimpan data yang dapat berubah ketika pengguna berinteraksi dengan aplikasi. Pada proyek ini, state digunakan untuk mengatur jumlah kehadiran mahasiswa.

Contoh kode dari PenghitungKehadiran.jsx:

```
function PenghitungKehadiran() {
  const [jumlah, setJumlah] = useState(0);

  function tambah() {
    setJumlah((nilai) => Math.min(30, nilai + 1));
  }

  function kurangi() {
    setJumlah((nilai) => Math.max(0, nilai - 1));
  }

  function reset() {
    setJumlah(0);
  }
```

useState(0) membuat state dengan nilai awal 0. Fungsi setNilai digunakan untuk mengubah nilai tersebut. Fungsi tambah membatasi nilai maksimal menjadi 30, fungsi kurang membatasi nilai minimal menjadi 0, sedangkan reset mengembalikan nilai menjadi 0.

3. Key

Key berfungsi memberikan identitas unik pada setiap elemen dalam daftar yang ditampilkan menggunakan React. Key membantu React mengenali elemen ketika daftar diperbarui.

Contoh kode pada daftar mata kuliah:

```
return (
  <section className="kartu">
    <h2>Daftar Mata Kuliah</h2>
    <div className="daftar-kartu">
      {daftarKuliah.map((kuliah) => (
        <KartuMataKuliah
          key={kuliah.id}
          nama={kuliah.nama}
          kode={kuliah.kode}
          sks={kuliah.sks}
        />
      ))}
    </div>
  </section>
);
}
```

Pada kode tersebut, key={kuliah.id} menggunakan ID setiap mata kuliah sebagai identitas unik. Dengan demikian, React dapat mengenali setiap elemen dalam daftar ketika terjadi perubah

4. Event

Event digunakan untuk menjalankan fungsi ketika pengguna melakukan suatu tindakan, seperti menekan tombol atau mengisi input. Pada kode PenghitungKehadiran.jsx, event digunakan untuk menjalankan fungsi tambah, kurangi, dan reset.

Contoh kode:

```
return (
  <section className="kartu">
    <h2>Penghitung Kehadiran</h2>
    <div className="angka-kehadiran">{jumlah}</div>
    <p>Batas nilai: 0 sampai 30</p>
    <div className="aksi">
      <button onClick={kurangi}>Kurangi</button>
      <button onClick={tambah}>Tambah</button>
      <button onClick={reset}>Reset</button>
    </div>
  </section>
);
}
```

onClick merupakan event yang menjalankan fungsi ketika tombol diklik. Misalnya, tombol Tambah menjalankan fungsi tambah untuk menambah jumlah kehadiran.

F. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa React dapat digunakan untuk membangun aplikasi web yang interaktif dan terstruktur melalui penggunaan JSX dan komponen. Konsep props diterapkan untuk mengirimkan data antarkomponen, sedangkan state digunakan untuk mengelola data yang dapat berubah sesuai interaksi pengguna. Selain itu, event handling memungkinkan tombol menjalankan fungsi tertentu, dan penggunaan key membantu React mengenali setiap elemen dalam daftar.

Penerapan konsep tersebut terlihat pada fitur profil mahasiswa, daftar mata kuliah, penghitung kehadiran, dan daftar tugas. Penghitung kehadiran memiliki batas nilai dari 0 hingga 30, sedangkan daftar tugas menyediakan fitur penambahan, penyaringan berdasarkan status, dan pengeditan judul. Penggunaan localStorage juga memungkinkan data tugas tetap tersimpan ketika halaman dimuat ulang, sementara validasi input membantu mencegah penyimpanan judul tugas yang kosong.

Melalui praktikum ini, pemahaman mengenai dasar pengembangan aplikasi menggunakan React menjadi lebih baik, mulai dari menyusun komponen hingga mengelola data dan interaksi pengguna. Praktikum ini juga memberikan pengalaman dalam memasang dependensi, menjalankan aplikasi menggunakan Vite, serta mendokumentasikan proyek agar dapat dijalankan kembali oleh orang lain.

G. Refleksi

1. Perbedaan props dengan state

Props adalah data yang dikirim dari komponen induk ke komponen anak. Props digunakan untuk memberikan informasi kepada komponen. Sementara itu, state adalah data yang dikelola di dalam komponen dan dapat berubah sesuai interaksi pengguna. Contohnya, props digunakan untuk mengirim nama mata kuliah ke komponen kartu, sedangkan state digunakan untuk menyimpan daftar tugas yang ditambahkan pengguna.

2. Mengapa map memerlukan key?

Key digunakan untuk memberikan identitas unik pada setiap elemen yang ditampilkan dari array menggunakan map(). Dengan adanya key, React dapat mengenali elemen yang ditambahkan, dihapus, atau diubah sehingga pembaruan tampilan menjadi lebih tepat.

3. Mengapa handler form memakai preventDefault?

preventDefault() digunakan untuk mencegah perilaku bawaan form, yaitu memuat ulang halaman ketika tombol submit ditekan. Dengan begitu, data dapat diproses oleh React tanpa harus me-refresh halaman.

4. Mengapa array diperbarui dengan map dan filter?

map() digunakan untuk membuat array baru dengan mengubah elemen tertentu, sedangkan filter() digunakan untuk membuat array baru yang hanya berisi elemen yang memenuhi kondisi tertentu. Cara ini membantu memperbarui data di React tanpa mengubah array asli secara langsung.

5. Mengapa data versi dasar hilang setelah refresh?

Data pada versi dasar disimpan di state atau memori aplikasi, bukan di penyimpanan permanen. Ketika halaman di-refresh, aplikasi dimuat ulang dan state kembali ke nilai awal sehingga perubahan data sebelumnya hilang. Agar data tetap tersimpan setelah refresh, data dapat disimpan menggunakan localStorage atau database.